

Aprendizaje Automático Probabilístico

Redes neuronales para imágenes

(actividades)



Alfons Juan
DSIC

Departamento de Sistemas
Informáticos y Computación

Índice

14.1	Introducción	1
14.2	Capas comunes	2
14.2.1	Capas convolucionales	2
14.2.2	Capas de agrupación	10
14.2.3	Todo junto	11
14.2.4	Capas de normalización	12
14.3	Arquitecturas de clasificación de imágenes	13
14.4	Otras formas de convolución	16
14.5	CNNs para otras tareas de visión	18

14.1. Introducción

► Los MLPs no se aplican directamente a imágenes. En su lugar, se suelen emplear redes neuronales convolucionales (CNNs), que sustituyen el producto matricial de los MLPs por una convolución. En relación con las CNNs, indica la respuesta incorrecta:

1. La imagen se divide en patches solapados y cada uno se compara con un conjunto de matrices de pesos pequeños, o filtros, que representan partes de un objeto

2. Se puede ver como una forma de template matching

3. Los filtros suelen ser pequeños (3×3 o 5×5) y se aprenden a partir de los datos, por lo que vienen a ser una extracción de características invariante a traslaciones integrada en el modelo

4. Todas son correctas

14.2. Capas comunes

14.2.1. Capas convolucionales

- Dados $x = [x_0, x_1, x_2, x_3] = [1, 2, 3, 4]$ y $w = [w_0, w_1, w_2] = [7, 6, 5]$, halla su convolución (discreta) y correlación cruzada en $i = 2$, $[w \otimes x](2)$ y $[w * x](2)$.



► Reproduce el cuaderno de la [Figura 14.5](#) para hallar:

$$\mathbf{Y} = \begin{bmatrix} w_1 & w_2 \\ w_3 & w_4 \end{bmatrix} \otimes \begin{bmatrix} x_1 & x_2 & x_3 \\ x_4 & x_5 & x_6 \\ x_7 & x_8 & x_9 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix} \otimes \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix} = \begin{bmatrix} 19 & 25 \\ 37 & 43 \end{bmatrix}$$

Más directamente, prueba el siguiente script:

nmimg14-2p-1-2.py

```
import jax
import jax.numpy as jnp

def corr2d(x: jnp.ndarray, k: jnp.ndarray) -> jnp.ndarray:
    """Compute 2d cross-correlation."""
    h, w = k.shape
    y = jnp.zeros((x.shape[0] - h + 1, x.shape[1] - w + 1))
    for i in range(y.shape[0]):
        for j in range(y.shape[1]):
            y = y.at[i, j].set(jnp.sum(x[i : i + h, j : j + w] * k))
    return y

if __name__ == '__main__':
    x = jnp.array([[0.0, 1.0, 2.0], [3.0, 4.0, 5.0], [6.0, 7.0, 8.0]])
    k = jnp.array([[0.0, 1.0], [2.0, 3.0]])
    print(corr2d(x, k))
```

nmimg14-2p-1-2.out

```
[[19. 25.]
 [37. 43.]]
```

► Aunque las CNNs sustituyen el producto matricial de los MLPs por una convolución, en realidad la convolución es un operador lineal que puede representarse mediante multiplicación matricial. En relación con esta afirmación, indica la respuesta correcta.

1. Es falsa ya que la convolución es un operador no lineal

2. Es falsa ya que la convolución es un operador lineal, pero no puede representarse mediante multiplicación matricial

3. Es cierta y, de hecho, las CNNs no son más que MLPs con matrices de pesos de estructura dispersa especial y elementos ligados en diferentes posiciones

4. Es cierta y, de hecho, las CNNs no son más que MLPs con matrices de pesos densas y elementos independientes

► Dada una imagen de alto-por-ancho $x_h \times x_w$, un filtro $f_h \times f_w$, y relleno p_h y p_w , el tamaño de la salida es:

$$1. (x_h + p_h - f_h + 1) \times (x_w + p_w - f_w + 1)$$

$$2. (x_h + p_h - f_h) \times (x_w + p_w - f_w)$$

$$3. (x_h + 2p_h - f_h + 1) \times (x_w + 2p_w - f_w + 1)$$

$$4. (x_h + 2p_h - f_h) \times (x_w + 2p_w - f_w)$$

► Sean una imagen de alto-por-ancho $x_h \times x_w = 7 \times 9$, un filtro $f_h \times f_w = 3 \times 3$, relleno $p_h = p_w = 1$ y saltos $s_h = s_w = 2$. Determina el tamaño de la salida.



- Reproduce el cuaderno de la [Figura 14.9](#) para hallar una convolución con múltiples canales de entrada o prueba:

```

1 import jax
2 import jax.numpy as jnp
3 from nimg14_2p_1_2 import corrd
4
5 def corrd_mult_in(X: jnp.ndarray, K: jnp.ndarray) -> jnp.ndarray:
6     # Iterate through the 0th dim (channel) of X and K. Then, add.
7     return sum(corrd(x, k) for x, k in zip(X, K))
8
9 if __name__ == '__main__':
10     x = jnp.array([
11         [1.0, 1.0, 2.0], [3.0, 4.0, 5.0], [6.0, 7.0, 8.0],
12         [1.0, 2.0, 3.0], [4.0, 5.0, 6.0], [7.0, 8.0, 9.0],
13         [1.0, 2.0, 3.0], [2.0, 3.0], [1.0, 2.0], [3.0, 4.0]])
14     print(x.shape) # 2 channels, each 3x3
15     print(k.shape) # 2 sets of 2x2 filters
16     out = corrd_mult_in(x, k)
17     print(out.shape)
18     print(out)

```

nimg14_2p_1_6.out

```

(2, 3, 3)
(2, 2, 2)
(2, 2)
[[ 56.  72.]
 [104. 120.]]

```

- Reproduce el cuaderno de la [Figura 14.9](#) para hallar una convolución con múltiples canales de entrada y salida, o prueba:

nmimg14-2p-1-bb.py

```

1 import jax
2 import jax.numpy as jnp
3 from nmimg14-2p-1-2 import corrd
4 from nmimg14-2p-1-6 import corrd_multt_in
5
6 def corrd_multt_in_out(X: jnp.ndarray, K: jnp.ndarray) -> jnp.ndarray:
7     # Iterate through the 0th dimension of `K`, and each time, perform
8     # cross-correlation operations with input `X`. All of the results are
9     # stacked together
10    return jnp.stack([corrd_multt_in(X, k) for k in K], 0)
11
12 if __name__ == '__main__':
13     X = jnp.array([
14         [1.0, 1.0, 2.0], [3.0, 4.0, 5.0], [6.0, 7.0, 8.0],
15         [1.0, 2.0, 3.0], [4.0, 5.0, 6.0], [7.0, 8.0, 9.0], [
16     K = jnp.array([[[0.0, 1.0], [2.0, 3.0]], [[1.0, 2.0], [3.0, 4.0]]])
17     K = jnp.stack((K, K + 1, K + 2), 0)
18     print(K.shape)
19     out = corrd_multt_in_out(X, K)
20     print(out.shape)

```

nmimg14-2p-1-bb.out

```

(3, 2, 2, 2)
(3, 2, 2)

```

(2, 3, 3) — nm1mg14-2p-1-7.out

```
import jax
import jax.numpy as jnp

def corr2d_multii_in_out_1x1(X: jnp.ndarray, K: jnp.ndarray) -> jnp.ndarray:
    # 1x1 conv is same as multiplying each feature column at each pixel
    # by a fully connected matrix
    c_i, h, w = X.shape
    c_o = K.shape[0]
    X = X.reshape((c_i, h * w))
    K = K.reshape((c_o, c_i))
    Y = jnp.matmul(K, X) # Matrix multiplication in the fully-connected layer
    return Y.reshape((c_o, h, w))

if __name__ == '__main__':
    from nimg14_2p1_6b import corr2d_multii_in_out
    key = jax.random.PRNGKey(1)
    X = jax.random.truncated_normal(key, 0, 1, (3, 3, 3)) # 3 channels per pixel
    K = jax.random.truncated_normal(key, 0, 1, (2, 3, 1, 1)) # map from 3 channels to 2
    Y1 = corr2d_multii_in_out_1x1(X, K)
    Y2 = corr2d_multii_in_out(X, K)
    print(Y2.shape)
    assert float(jnp.abs(Y1 - Y2).sum()) > 1e-6
```

nnim914-2p-1-7. py

ción txt o prueba:

► Reproduce el cuaderno de la **Figura 14.9** para hallar una convolu-

14.2.2. Capas de agrupación

► Reproduce el cuaderno de la [Figura 14.9](#) para pooling o prueba:

```
import jax
import jax.numpy as jnp

def pool2d(X: jnp.ndarray, pool_size: tuple[int], mode: str = "max") ->
    jnp.ndarray:
    p_h, p_w = pool_size
    Y = jnp.zeros((X.shape[0] - p_h + 1, X.shape[1] - p_w + 1))
    for i in range(Y.shape[0]):
        for j in range(Y.shape[1]):
            if mode == "max":
                Y = Y.at[i, j].set(X[i : i + p_h, j : j + p_w].max())
            elif mode == "avg":
                Y = Y.at[i, j].set(X[i : i + p_h, j : j + p_w].mean())
    return Y

if __name__ == '__main__':
    X = jnp.arange(16).reshape((4, 4))
    print(X)
    print(X.shape)
    print(pool2d(X, (3, 3), "max"))
```

nmimg14-2p-2.out

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
[12 13 14 15]]
(4, 4)
[[10. 11.]
 [14. 15.]
```

14.2.3. Todo junto

► La arquitectura clásica de las CNNs se inspira en el Neocognitron de Fukushima. En esencia, esta arquitectura:

1. Alterna capas convolucionales con max pooling, más una capa de clasificación lineal al final
2. Alterna capas convolucionales con avg pooling, más una capa de clasificación lineal al final
3. Aplica capas convolucionales sucesivas, más una capa de max pooling y otra de clasificación lineal al final
4. Aplica capas convolucionales sucesivas, más una capa de avg pooling y otra de clasificación lineal al final



14.2.4. Capas de normalización

► La normalización consiste en añadir capas extra para evitar el problema de los gradientes que desaparecen o explotan en redes profundas. La técnica más popular, la normalización batch, normaliza la distribución de las activaciones de una capa para que tenga media nula y varianza unitaria a nivel de minibatch. En relación con esta técnica, indica la respuesta incorrecta.

1. En aprendizaje se recalculan medias y varianzas en cada minibatch ya que los parámetros van cambiando

2. En inferencia se usan medias y varianzas estimadas a partir de todos los datos tras finalizar el aprendizaje

3. Puede dar lugar a estimadores inestables de los parámetros cuando se entrena con tamaños de batch pequeños

4. Todas son correctas



14.3. Arquitecturas de clasificación de imágenes

► La clasificación de imágenes en tareas populares, especialmente ImageNet, ha estimulado el desarrollo de CNNs cada vez más precisas. En relación con ellas, indica la respuesta incorrecta.

1. LeNet (1998), creada por Yann LeCun para MNIST, obtuvo una notable mejora de precisión frente a los MLPs convencionales

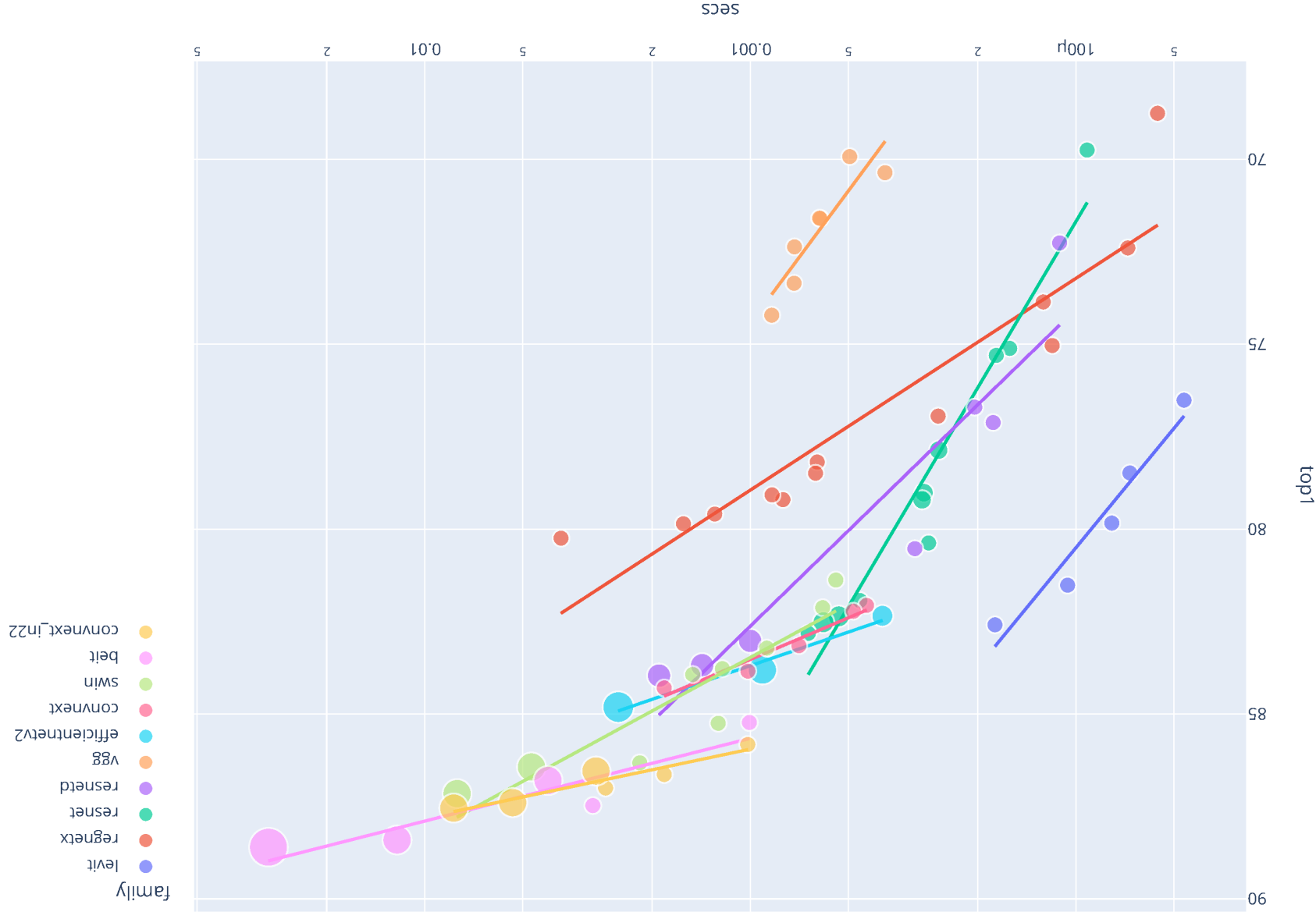
2. AlexNet (2012), desarrollada por Alex Krizhevsky para ImageNet, redujo el error del 26 % al 15 %

3. Resnet (2015), propuesta por un equipo de Microsoft para ImageNet, obtuvo tasas de error super-humanas ($< 5\%$) mediante el uso de bloques residuales

4. Todas son correctas



► **Pytorch (torchvision)** y la librería **timm** facilitan el seguimiento y uso de arquitecturas SOTA para clasificación de imágenes. Re-produce el cuaderno kaggle “Which image models are best?”:



► **fastai**: librería de alto nivel para la construcción rápida de sistemas SOTA en dominios de aprendizaje profundo estándar; en particular, facilita el fine-tuning de modelos típicos en tareas downs-tream. Trata de ejecutar y averiguar qué hace el script:

```

1 from fastai.vision.all import *
2 path = untar_data(URLs.PETS) / 'images'
3 dls = ImageDataLoaders.from_name_func(
4     path, get_image_files(path), valid_pct=0.2,
5     label_func=lambda x: x[0].isupper(), item_tfms=Resize(224))
6 learn = vision_learner(dls, 'vit_tiny_patch16_224',
7     ↪ metrics=error_rate)
8 learn.fine_tune(1)

```

epoch	train_loss	valid_loss	error_rate	time
0	0.159362	0.015760	0.004736	00:13
epoch	train_loss	valid_loss	error_rate	time
0	0.033591	0.009284	0.002706	00:14

Ver el computer vision intro (single-label classification) de fastai

14.4. Otras formas de convolución

► La convolución estándar se extiende a otros tipos de convoluciones para abordar aplicaciones tales como la segmentación y generación de imágenes. En relación con estas extensiones, indica la respuesta incorrecta.

1. Dado un factor o ratio de dilatación r , la convolución dilatada, convolución con agujeros o algoritmo à trous, toma cada r -ésimo elemento de la entrada

2. Al contrario que la convolución convencional, la transpuesta produce una salida grande a partir de una entrada pequeña

3. La convolución separable en profundidad convoluciona cada canal de entrada con su correspondiente filtro y luego transforma los C canales resultantes en D mediante convolución 1×1

4. Todas son correctas



► Consulta la [computer vision intro \(segmentation\)](#) de fastai y trata de ejecutar y averiguar qué hace el script:

```

1 from fastai.vision.all import *
2 path = untar_data(URLs.CAMVID_TINY)
3 codes = np.loadtxt(path/'codes.txt', dtype=str)
4 fnames = get_image_files(path/'images')
5 def label_func(fn): return path/"labels"/f"{fn.stem}_{fn.suffix}"
6 dls = SegmentationDataLoaders.from_labels_func(
7     path, bs=8, fnames = fnames, label_func = label_func, codes = codes)
8 learn = unet_learner(dls, resnet34)
9 learn.fine_tune(6)

```

```

1 epoch      train_loss      valid_loss      time
2 0           3.246676      2.349724      00:03
3 epoch      train_loss      valid_loss      time
4 0           1.977266      1.573830      00:01
5 1           1.662492      1.352440      00:01
6 2           1.483450      1.270014      00:02
7 3           1.362093      0.972190      00:01
8 4           1.245346      0.904833      00:02
9 5           1.152846      0.885898      00:01

```

14.5. CNNs para otras tareas de visión

► Aparte de la clasificación de imágenes (con una sola etiqueta), las CNNs se emplean en otras tareas de visión. En relación con estas otras tareas, indica la respuesta incorrecta.

1. Una extensión usual de la clasificación mono-etiqueta es la multi-etiqueta; en tal caso se habla de etiquetado de imágenes

2. La detección de objetos produce cajas de mínima inclusión donde se hallan los objetos de interés, con sus etiquetas de clase

3. La segmentación de instancias y la semántica son tareas parecidas; la primera produce una máscara de forma 2d y una etiqueta de clase por cada instancia de objeto en la imagen, mientras que la segunda produce una etiqueta de clase por cada píxel (sin diferenciar objetos)

4. Todas son correctas

